Lecture 2: Formal Models of AI Problems and Search

Viliam Lisý & **Branislav Bošanský**

Artificial Intelligence Center
Department of Computer Science, Faculty of Electrical Eng.
Czech Technical University in Prague

bosansky@fel.cvut.cz

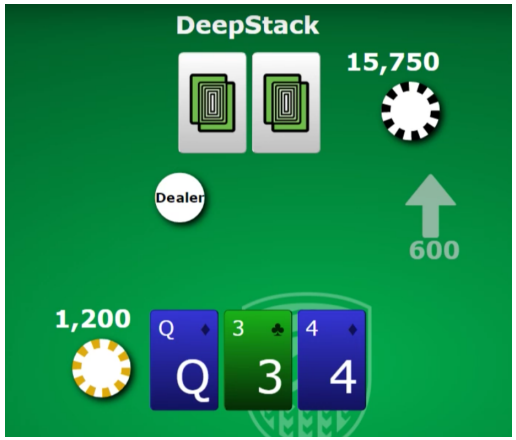
February, 2026

Questions for the Lecture

- Why it is good to have a formal representation of a problem?
- Why we are talking about search at all?
- Provide a unifying perspective on different search algorithms.

Many (even recent) great AI breakthroughs use search as one of the components.







Current AI and Formal Representation

Nowadays, large language models are being used as a synonym for AI $\rightarrow$ they are using natural language as an input

- Do we really need some other representation for solved problems?
- What are the disadvantages of natural language?
- Natural language is ambiguous (words may have different meanings)
  - What does goal 'collect all gold' for a robot mean? Can it carry all the gold at once or should they be collected one-by-one? How close the robot needs to be to a gold in order to collect it? ...

Formal representation of problems reduces ambiguity and allows us to design algorithms with guaranteed outcomes.

Test-Time Compute: LLMs Think Harder with Search

Search underlies one of the most significant AI paradigm shifts in 2025:

- Models trained with **reinforcement learning with verifiable rewards** (RLVR) spontaneously develop strategies that resemble search – breaking problems into intermediate steps, exploring alternatives, backtracking.
- **OpenAI o3** (2025): a private chain of thought enables planning and multi-step reasoning; more compute for "thinking" $\Rightarrow$ better performance.
- **DeepSeek-R1** (Jan 2025): RL on a base model (no supervised fine-tuning) naturally produces *reflection* and extended reasoning – emergent search behavior.

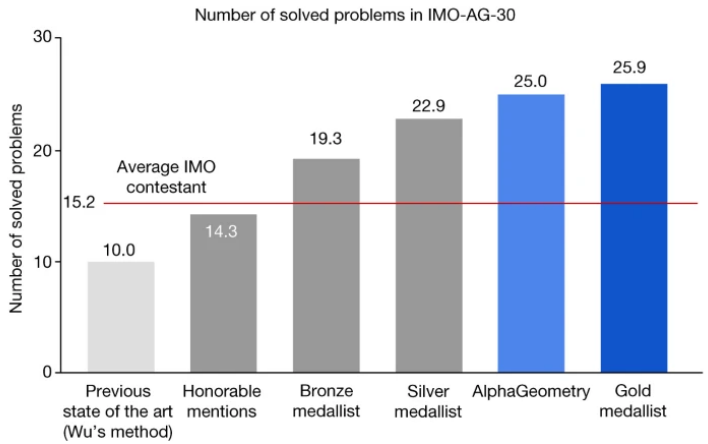
Connection to this lecture

Chain of thought = sequence of **states**. Each step = **action**. Verification = **goal test**. These models perform BFS / DFS / MCTS through reasoning space.

Formal Languages + Search = Provably Correct Results

Combining LLMs with **formal representations** and **search** enables verifiable outcomes:

Fig. 2: AlphaGeometry advances the current state of geometry theorem prover from below human level to near gold-medallist level.



- **AlphaProof** (Nature, 2025): AlphaZero-style RL with formal Lean proofs. Solved **4/6 IMO 2024** problems, including the hardest one (only 5 of 609 human participants solved it).
- **Gemini Deep Think** (IMO 2025): solved 5/6 problems at **gold-medal level** by simultaneously exploring multiple solution paths.

Why formal representation matters

Formal languages enable **step-by-step verification**. Natural language $\Rightarrow$ plausible but incorrect reasoning (hallucination). Formalization + search $\Rightarrow$ guaranteed correctness.

Search for Algorithm Discovery and AI Agents

Search is now used to **discover new algorithms** and to power **autonomous agents**:

- **AlphaEvolve** (DeepMind, May 2025): evolutionary coding agent searching through program space. Discovered a matrix multiplication procedure for 4×4 complex matrices using 48 scalar multiplications – **first improvement over Strassen's algorithm in 56 years**.
- **LLM Inference via Search** (TMLR 2025): BFS, DFS, MCTS, and Beam Search are actively used inside LLMs for reasoning, code generation, and web navigation.
- **Coding agents** (e.g., Claude Code): navigate states (current codebase), define a goal, prepare a plan, take actions (edit, run, read errors, fix), verify a goal (passing tests)

All examples share the same structure: **States** · **Actions** · **Goal** · **Search**.

Search and AI

Search is one of the most fundamental and universal methods for solving problems.

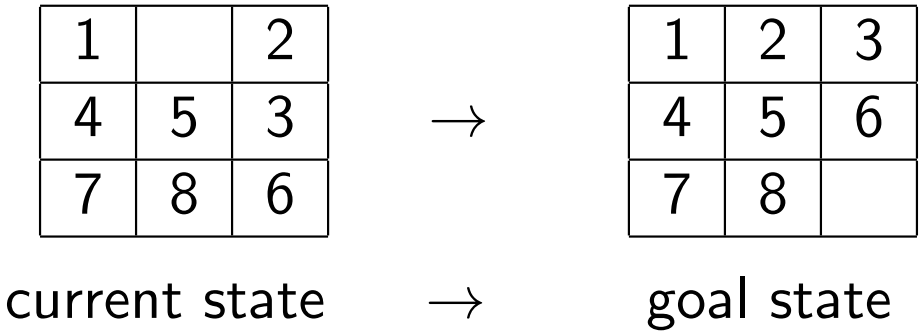
Having a **formal representation** of the problem, search algorithms allow us to look for a solution in a systematic way:

- formal representation of every possible situation in the scenario – the **states** of the problem (denoted S)
- how the states can be changed by the algorithm (agent) – the **actions** in the scenario (denoted A)

By applying an action $a \in A$ to a state $s \in S$, the state will change to a different state $s' \in S$.

States must contain all information necessary to determine applicable actions, transformation of the environment, or whether the goal has been reached.

Example 1 – 8 puzzle



Goal: rearrange the numbers by moving the empty square to adjacent squares so that the numbers are ordered

Possible representations:

- values of tiles in a sequence
  $s = [1, _, 2, 4, 5, 3, 7, 8, 6]$
- position of numbers
  $s = [1, 3, 6, 4, 5, 9, 7, 8]$

Example 2 – Robotic arm



Find correct configurations of joints / parts of the arm so that the arm catches a desired object.

Possible representations:

- $s = [\theta_1, \alpha_1, \theta_2, \alpha_2, \ldots]$

Example 3 – Chess



Possible representations:

- positions of pieces on the board
  $s = [[A1, B1, C1, \ldots, H2], [A8, \ldots, H7], \ldots] \rightarrow$ additional information needed besides the board itself (king has moved, rook has moved, repeated positions (!))

Alternatively, a **history of played moves** represents a state.

Solution of a Problem

Many of the AI problems can be formulated as finding a sequence of actions that leads to a **goal state**.

We want to find the best such a sequence

- minimize the number of actions
- every action can have some cost (or reward) associated with it
  $\rightarrow$ minimization of total cost (maximization of total reward)

We can reason about possible states / effects of actions
(the rules of the environment are known (!)):

- we have a formal model
- (for the large scale) access to a simulator

Main AI Models for (Sequential) Decision Making

Each problem can use a specific formal representation, however, there are several fundamental formal models when searching for optimal sequence of actions based on searching through state space (possibly uncertain effect of actions / stochastic environment):

- Markov Decision Processes (MDPs)
- Partially Observable Markov Decision Processes (POMDPs)
- (Imperfect Information) Extensive-Form Games (EFGs)
- (Partially Observable) Stochastic Games (POSGs)

We introduce selected general models now to emphasize the importance of the correct formalization of the problem. Selected algorithms for solving them optimally will be introduced later.

Main AI Models for (Sequential) Decision Making

There are several fundamental models when searching for optimal sequence of actions based on searching through state space (possibly uncertain effect of actions / stochastic environment):

- Markov Decision Processes (MDPs) $\rightarrow$ perfectly observable environment, only 1 agent is acting
- Partially Observable Markov Decision Processes (POMDPs) $\rightarrow$ **partially observable environment**, only 1 agent is acting
- (Imperfect Information) Extensive-Form Games (EFGs) $\rightarrow$ perfectly (partially) observable environment, finite horizon, n **agents can act (every agent optimizes own goal / utility)**
- (Partially Observable) Stochastic Games (POSGs) $\rightarrow$ perfectly (partially) observable environment, **infinite horizon**, n agents can act (every agent optimizes own goal / utility)

Markov Decision Processes (MDPs)

Consider (finite) sets of states S, rewards R, and actions A. The agent interacts with the environment in discrete steps $t = 0, 1, 2, \ldots$. At each timestep, the agent receives the current state $S_t \in S$, selects an action based on the state $A_t \in A$. As a consequence of taking the action, the agent receives a reward $R_{t+1} \in R$ and finds itself in a new state S_{t+1}.

Rewards and states are generated based on a *dynamics* of the MDP

$$p(s', r|s, a) \leftarrow Pr\{S_t = s', R_t = r|S_{t-1} = s, A_{t-1} = a\}$$

The next state depends only on the current state and the action (Markov property). **In the first lectures/labs, we assume that the environment is deterministic.**

Transition and reward dynamics can be defined separately.

Markov Decision Processes (MDPs) – Example

#	#	#	#	#	#
#	G				#
#	#	#	#		#
#	↓	#	#		#
#					#
#	#	#	#	#	#

Consider a robot (↓) in a maze (# are walls), the arrow represents the direction the robot is facing, G is gold.
What are the states and actions?

- s $= (X, Y, d, G)$
- actions = (move_forward, move_backward, turn_left, turn_right)

MDP dynamics:

- $p((1, 1, \downarrow, \text{false}), 0 \mid (1, 2, \downarrow, \text{false}), \text{move_forward}) = 1$
- $p((1, 1, \downarrow, \text{false}), 0 \mid (1, 2, \downarrow, \text{false}), \text{move_backward}) = 0$
- ...

Markov Decision Processes (MDPs)

Formalization using an MDP gives us:

- complete and unambiguous description of the problem
- well-defined inputs and outputs for the problem-solving algorithm
- the ability to reason about the problem
- possibility to use existing algorithms to find the solution

If we design and implement a brand-new algorithm for MDPs (POMDPs / EFGs / ...), we can solve (almost) all instances.

Partially Observable Markov Decision Processes (POMDPs)

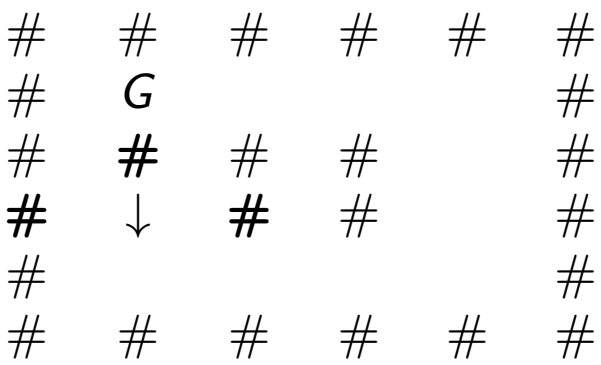
States, actions, and rewards are as before, however, the agent cannot perfectly observe the current state.

The agent has a **belief** – a probability distribution over states that expresses the (subjective) likelihood about the current state. The agent receives **observations** from a finite set O that affect the belief. The agent starts from an **initial belief** and based on actions and observations, it updates its belief. Given the current belief $b : S \to [0, 1]$ and some action $a \in A$ and received observation $o \in O$, the new belief is defined as:

$$b(s') = \mu O(o|s', a) \cdot \sum_{s \in S} Pr(s'|s, a) \cdot b(s)$$

where μ is a normalizing constant.

POMDP – Example



The robot can now perceive only its surroundings but does not know the exact position in the maze. States and actions remain the same.

- $s = (X, Y, d, G)$
- actions = (move_forward, move_backward, turn_left, turn_right)

Observations are all possible combinations of walls / free squares in the 4-neighborhood:

- $(\#, \#, \#, \#), (\#, \#, \#, _), \ldots$

Extensive-Form Games (EFGs)

Agent is not the only one that changes the environment. Every state has a player that acts in that state.EFGs are typically visualized as game-trees that:

- are finite (the game has some pre-defined horizon; note that (PO)MDPs do not have this!)
- node of the game tree corresponds to the history of actions from the beginning, edges are actions (as search trees)
- rewards (termed utilities) are defined only in terminal states (leaves of the game tree)
- agent can have imperfect information (certain states can be indistinguishable) $\rightarrow$ we will not be able to cover this in ZUI ($\rightarrow$ B4M36MAS)

Partially Observable Stochastic Games (POSGs)

POSGs are a multi-agent extension of POMDPs $\rightarrow$ every agent can have their own actions, observations, and rewards. Every agent has its own belief (about the state, about beliefs of other agents, ...).

One of the most general formal models $\rightarrow$ algorithmically intractable in general.

Solution of a Deterministic MDP

How can we find a solution of a deterministic MDP?

Find the best sequence of actions leading to the goal $\rightarrow$ explore relevant states of an MDP and find the best action to be played in these states such that the trajectory (or a run)

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots, S_k$$

maximizes the accumulated reward (and S_k is a goal state)[1].

For now, the rewards are summed together (in case of stochastic transitions / POMDPs / negative costs, a discounted sum is typically used with discount factor $0 < \gamma < 1$).

[1] Maximization of rewards = minimization of costs (we will use both).

Solving Deterministic MDPs

1. Start from the initial state S_0
2. Apply available actions to the current state and generate new possible states
3. Select one of the newly generated states as the current one
4. If the current state is the goal state $\rightarrow$ finish
5. If not, go to step 2

Questions:

- Which state should we select out of all generated new states in step 3?
- What if we generate a state that we have already explored?

Solving Deterministic MDPs – Choosing Next State

Q1

Which state should we select out of all generated new states in step 3?

Goal is to find the best sequence of actions $\rightarrow$ we want to explore the ones with the highest rewards (lowest costs) first.

What if we make a mistake? $\rightarrow$ We cannot forget which other states are reachable $\rightarrow$ We keep a (sorted) list of reachable states that can be further explored – **open list** or **fringe**.

Solving Deterministic MDPs – Variants of Uninformed Search

Variants of using the **fringe**:

- the fringe is sorted, new states to explore are taken from the beginning $\rightarrow$ **uniform-cost search**
- the fringe is unsorted, newly expanded states are inserted to the front, new states to explore are taken from the beginning $\rightarrow$ **depth first search (DFS)**
- the fringe is unsorted, newly expanded states are appended at the back, new states to explore are taken from the beginning $\rightarrow$ **breadth first search (BFS)**

BFS is complete, finds the shallowest solution (the sequence that requires the least number of actions while ignoring rewards). Requires exponential memory (and time).

Solving Deterministic MDPs – Variants of Uninformed Search

DFS is not complete (the algorithm might not terminate) $\rightarrow$ we limit the maximal length of the sequence of actions DFS can explore and iteratively increase this limit $\rightarrow$ **iterative deepening**.

Uniform-cost search is complete and optimal (in case all rewards are strictly negative). A variant of **Dijkstra's algorithm** (only the best path to a goal state not all states).

Solving Deterministic MDPs

Q2

What if we generate a state that we have already explored?

Using this algorithm, we are generating a **search tree**. Every node of the search tree corresponds to a state in the environment but multiple nodes can correspond to the same state.

We can maintain a **closed list** of already evaluated states.

Iterative Deepening

Combining good characteristics of BFS and DFS. Let's have a `limited-depth-dfs` method:

- call `limited-depth-dfs` with depth limit 0,
- if unsuccessful, call `limited-depth-dfs` with depth limit 1,
- if unsuccessful, call `limited-depth-dfs` with depth limit 2, etc.

Complete, finds the shallowest solution, space requirements of a DFS.Counterintuitively, it is not that wasteful (timewise):

- the search tree grows exponentially $\rightarrow$ it is more time consuming to generate / evaluate all states in depth exactly d than repeatedly visiting states in the shallower depth

What if we want to optimize cost instead of number of actions? $\rightarrow$ limit the overall cost and increase the cost iteratively by 1.

Summary and Outlook

Revisiting the opening questions:

- *Why is formal representation useful?* – Eliminates ambiguity; enables algorithms with provable guarantees; powers state-of-the-art AI (reasoning models, AlphaProof, coding agents).
- *Why search?* – Finding the optimal sequence of actions *is* search through a state space.
- *A unifying perspective on search algorithms?* – All variants (BFS, DFS, uniform-cost, iterative deepening) differ only in how they **order the fringe**.

Next lecture

All algorithms today treated every state equally – they have no way to judge which states are more *promising*.

What if the algorithm could **estimate how far it is from the goal**? $\Rightarrow$ dramatically more efficient search.